YouTube DataOps & Analytics Platform (2026 Edition)

Enterprise Architektur-Handbuch, Technisches Repertorium & Deep-Dive Dokumentation

Von Bare-Metal-Hadoop zu Serverless Cloud Lakehouse, Apache Iceberg V2 & DataOps

---

Dokumenten-Typ:	Enterprise Architecture Reference, Technical Repertorium & Operations Manual
Autor & Lead Architect:	Bernd Weltler
Aktuelle Version:	Release `v2.1.0` (Serverless Cloud & Observability) — inkl. `v1.0.0` & `v2.0.0` Historie
Status:	Produktionsreif (100% Deployed, Getestet & Verifiziert)
GCP Projekt-ID:	`cloudanalytics-514018` (Region: `europe-west3` Frankfurt / Location: EU)
Service Account:	`cloudanalytics-514018@cloudanalytics-514018.iam.gserviceaccount.com`
GitHub Repository:	`https://github.com/BHW80/DE_Capstone_2026.git`
Technologie-Stack:	Apache Kafka KRaft, Apache Spark 3.5, Apache Iceberg V2, dbt Core 1.13, Google BigQuery, Terraform 1.17, Google Cloud Workflows, Grafana Cloud, GitHub Actions

Executive Scope: Dieses Handbuch dient als lückenloses Referenzwerk, Betriebshandbuch und Repertorium für die vollständige Modernisierung der YouTube Trending Video Data Pipeline. Es dokumentiert jede Architekturentscheidung (ADRs), die Migration von fragilen On-Premises-Altsystemen auf Open Table Formats (Apache Iceberg V2), den Aufbau eines dimensionalen Data Warehouse nach Ralph Kimball (SCD-2 via dbt Core), das Infrastructure-as-Code-Design mit Terraform 1.17, den 0-€ Kostenvorteil serverloser Cloud Workflows, das cloud-native Observability-Konzept via Grafana Cloud sowie die GitHub Actions CI/CD-Automatisierung. Zusätzlich grenzt es für alle drei Projekt-Releases (`v1.0.0`, `v2.0.0`, `v2.1.0`) präzise ab, welche Komponenten zwingend erforderlich sind und welche historisch abgelöst wurden.

Inhaltsverzeichnis

1. Management Summary & Strategisches Leitbild	Seite 3
2. Die Evolutionsstufen & Release-Notwendigkeits-Matrix (v1.0 / v2.0 / v2.1)	Seite 4
3. End-to-End Systemarchitektur, Medallion Lakehouse & Repository-Struktur	Seite 7
4. Streaming Ingestion & Resilienz (Kafka KRaft, DLQ & SCD-2 Mutation)	Seite 9
5. Ephemerer Compute & Open Table Format (Apache Iceberg V2)	Seite 11
6. Cloud Data Warehouse & Kimball Star-Schema (dbt Core & SCD-2)	Seite 13
7. DataOps Quality Gates: 15 dbt Tests & In-Workflow Validierung	Seite 15
8. Infrastructure as Code (Terraform 1.17 Cloud Provisioning)	Seite 16
9. Serverlose Orchestrierung (Google Cloud Workflows State-Machine)	Seite 18
10. Cloud-Native Observability (Grafana Cloud & GCP Monitoring - Weg 1)	Seite 20
11. DataOps CI/CD Pipelines, GitOps & Release-Management	Seite 21
12. Architectural Decision Records (ADRs 001 bis 006) im Detail	Seite 23
13. Deppensicheres Betriebshandbuch (9-Schritte-Anleitung & Playbooks)	Seite 25
14. Data Engineering Interview Masterclass & Technical Defense (15 Q&As)	Seite 29
15. FinOps, Kosten-Engineering & Always-Free-Tier Kapazitätsplanung	Seite 32

1. Management Summary & Strategisches Leitbild

Die moderne Datenlandschaft verlangt von Analytics-Plattformen heute weit mehr als das bloße Ausführen statischer SQL-Skripte oder nächtlicher Batch-Jobs. Unternehmen stehen vor der Herausforderung, kontinuierlich eintreffende Datenströme fehlertolerant aufzunehmen, fehlerhafte Events ohne Datenverlust zu isolieren, Datenänderungen historisch lückenlos nachzuvollziehen (SCD-2) und strenge analytische Qualitätsgarantien durchzusetzen — und das bei minimalen Betriebskosten.

Das vorliegende Projekt **YouTube DataOps & Analytics Platform (2026 Edition)** demonstriert die vollständige Transformation eines wartungsintensiven universitären Legacy-Systems (Post Graduate Program in Data Engineering, Purdue University / Simplilearn 2021) in eine cloud-native Enterprise-Lösung nach modernsten Data-Engineering- und DevOps-Standards.

Kernmetriken & Business Impact im Vergleich:

Dimension	Legacy-System (2021)	DataOps Platform (2026 v2.1)	Business & Tech Impact
Monatliche Infrastrukturkosten	Hohe VM-Kosten oder unkalkulierbare Cloud-Cluster	**0,00 € laufende Idle-Kosten** (Serverless Cloud Workflows & GCP Free Tier)	100% Kostensicherheit; keine 24/7 GKE- oder Composer-Gebühren (~300 €/Monat gespart)
Datenresilienz & Fehlertoleranz	Pipeline bricht bei fehlerhaften Records komplett ab	**Automatisierte Dead Letter Queue (DLQ)** Quarantäne (`topic_err`)	Zero Data Loss: 295 Gutdaten fließen weiter, 31 korrupte Records (~10%) werden isoliert
Transaktionssicherheit (ACID)	Flache Parquet-/CSV-Dateien (Gefahr von Teilabbrüchen)	**Apache Iceberg V2** Table Format (`local.silver.videos`)	Garantiert atomare Commits, Snapshot-Isolation, Manifest-Pruning & Time-Travel
Historisierung & Auditfähigkeit	SCD-1 (Alte Werte werden blind überschrieben)	**SCD-2 Historisierung** via dbt Snapshots & Point-in-Time Joins	Lückenlose Nachvollziehbarkeit mutierter Videotitel (`2kyS6SvSYSE`) über 3 Versionen
Quality Assurance & Governance	Keine automatisierten Datenprüfungen	**15 dbt Data Tests + In-Workflow Foreign-Key Gate**	Orphan Records und Schema-Drifts werden vor der BI-Freigabe hart blockiert (`raise`)
Deployment & Reproduzierbarkeit	Manuelle Windows-Klick-Anleitungen & Batch-Dateien	**Terraform 1.17 IaC + GitHub Actions CI/CD**	5 GCP-Ressourcen in 60 Sekunden reproduzierbar bereitgestellt und rückstandslos löschbar

Architektur-Prämisse:
Die fundamentale Prämisse dieser Plattform lautet: **Zero Idle Cost bei maximaler Produktionshärte**. Statt kostspielige Airflow-Cluster (Cloud Composer) 24/7 leerlaufen zu lassen, orchestriert ein serverloser Google Cloud Workflow BigQuery-Jobs und führt In-Database Quality Gates aus. Damit eignet sich dieses System als Blaupause für moderne Cloud Data Platforms in Startups wie im Enterprise-Umfeld.

2. Die Evolutionsstufen & Release-Notwendigkeits-Matrix

Um den technologischen Wert der aktuellen Architektur zu verstehen, muss man die vier Entwicklungsstufen des Projekts betrachten: vom fragilen 2021er On-Premises-Setup über das containerisierte Fundament (`v1.0.0`) und die Lakehouse/Kimball-Ausbaustufe (`v2.0.0`) bis hin zur serverlosen Cloud-Plattform (`v2.1.0`).

2.1 Die 2021 Legacy-Architektur (Historische Schmerzpunkte)

- **Windows OS-Inkompatibilität:** Lokale Hadoop/HDFS-Aufrufe scheiterten an Windows-Pfaden und erforderten inoffizielle `winutils.exe`-Binaries.
- **ZooKeeper als Single Point of Failure:** Separate ZooKeeper-Prozesse führten unter Windows zu Verbindungsabbrüchen, Session-Timeouts und blockierten Kafka-Brokern.
- **Fehlende Dead Letter Queue (DLQ):** Ein einziger korrupter Datensatz mit nicht-numerischen `views` brachte die gesamte Pipeline zum Absturz.
- **SCD-1 Blindüberschreibung:** Änderungen an Videotiteln wurden ohne Historie überschrieben. Punktgenaue Analysen vor/nach Titeländerungen waren unmöglich.
- **Manuelles ClickOps statt IaC/CI/CD:** Cloud-Ressourcen wurden per Mausklick angelegt; automatisierte Tests oder reproduzierbare Deployments existierten nicht.

2.2 Die Evolutionsmatrix: Vergleich aller 4 Generationen

Kriterium	2021 Legacy	v1.0.0 (Foundation)	v2.0.0 (Lakehouse)	v2.1.0 (Aktuell)
Bereitstellung	Bare Metal Windows	Docker Compose	Docker + Python `.venv`	**Terraform 1.17 IaC**
Streaming Engine	Kafka + ZooKeeper	Kafka KRaft (Zero-ZK)	KRaft + SCD-2 Events	**KRaft + GCP Pub/Sub**
Resilienz & Fehler	Keine (Jobabbruch)	Parquet Quarantäne	DLQ Consumer (`topic_err`)	**DLQ + Pub/Sub Error Topic**
Storage Format	Flache CSV-Dateien	Snappy Parquet	**Apache Iceberg V2**	**Iceberg V2 + GCS Bucket**
Datenmodellierung	Unstrukturiert	Flache Spark Gold Marts	**Kimball Star-Schema**	**Kimball + SCD-2 Marts**
Historisierung	Keine (SCD-1)	Keine	**SCD-2 dbt Snapshots**	**Point-in-Time Join DWH**
Quality Gates	Keine	Basis Row-Count	**15 dbt Data Tests**	**15 Tests + In-Workflow Gate**
Orchestrierung	Manuelle Batch-Skripte	Airflow 2.9 (Docker)	PowerShell Runner	**Serverless Cloud Workflows**
Observability	Keine	Lokaler Prometheus/Grafana	Lokaler Container-Stack	**Grafana Cloud (Weg 1)**
CI/CD & Releases	Keine	Git Tag `v1.0.0`	Git Tag `v2.0.0`	**GitHub Actions (`v2.1.0`)**

2.3 Notwendigkeits-Matrix über alle drei Releases (Was wird gebraucht – und was nicht?)

Ein entscheidendes Qualitätsmerkmal professioneller DataOps-Dokumentation ist die glasklare Abgrenzung zwischen **zwingend notwendigen Produktivkomponenten** und **historisch abgelöstem Legacy-Code (Deprecations)** in jedem der drei Git-Releases (`v1.0.0`, `v2.0.0`, `v2.1.0`):

Release	Zwingend notwendig (Must-Have)	Vollständig überflüssig / Nicht notwendig

Release v1.0.0 ***(Core Foundation)***	• Docker Desktop & Docker Compose • Apache Kafka im KRaft-Modus (`127.0.0.1:9092`) • Streaming Producer mit DLQ (`src/producer.py`) • Bronze & DLQ Consumer (`src/consumer_bronze.py`, `src/consumer_dlq.py`) • Ephemerer Docker PySpark (`src/transform_silver.py`, `src/transform_gold.py`) • Lokaler Airflow 2.9 & PostgreSQL Stack (`dags/`, `docker-compose-airflow.yml`) • Lokaler Prometheus & Grafana Stack (`docker-compose-monitoring.yml`)	• **Kein GCP Cloud Account zwingend nötig** (reines lokales Offline-Testing möglich) • **Kein BigQuery** (Speicherung rein lokal als Snappy Parquet) • **Kein dbt Core** (Transformationen rein in Spark Python) • **Kein Apache Iceberg** (nur lose Parquet-Dateien) • **Kein Terraform** (reine Docker Compose Verwaltung) • **Keine Cloud Workflows** (Scheduler ist lokaler Airflow) • **Keine Grafana Cloud** (lokaler Container auf Port 3000) • **Kein GitHub Actions CI/CD**
Release v2.0.0 ***(Lakehouse & Kimball Marts)***	• Python Virtual Environment (`.venv`) mit `requirements.txt` • GCP Projekt (`cloudanalytics-514018`) & Key (`gcp_service_account.json`) • GCP IAM-Rollen: `roles/bigquery.admin`, `roles/storage.admin` • Apache Iceberg V2 Runtime im Spark-Container (`iceberg-spark-runtime-3.5`) • Iceberg Warehouse (`data/iceberg_warehouse/silver/videos/`) • dbt Core Projekt (`dbt/`) mit BigQuery-Profil (`profiles.yml`) • dbt SCD-2 Snapshots (`dim_video_snapshot.sql`) • 15 automatisierte dbt Tests (`schema.yml`) • Idempotenter BigQuery Uploader (`src/cloud_loader_bigquery.py`, `load_to_bigquery.ps1`)	• **Altes Spark Gold Script (`src/transform_gold.py`, `run_gold.ps1`):** Nicht mehr nötig! Vollständig durch dbt Kimball Marts in BigQuery abgelöst. • **Lose Parquet-Dateien als alleiniger Silver-Layer:** Durch Apache Iceberg V2 mit ACID-Commits und Snapshot-Historie ersetzt. • **SCD-1 Überschreibung:** Abgelöst durch SCD-2 Snapshot Historisierung. • **Noch nicht vorhanden / nicht nötig:** Terraform, Cloud Workflows, Pub/Sub Topics, Grafana Cloud, GitHub Actions CI/CD.
Release v2.1.0 ***(Serverless Cloud & Observability)*** **[AKTUELLE PRODUKTION]**	• Terraform 1.17 (`run_infra.ps1`) • GCP Service Account mit 6 IAM-Rollen (`bigquery.admin`, `storage.admin`, `pubsub.admin`, `workflows.admin`, `iam.serviceAccountUser`, `monitoring.viewer`) • 5 aktivierte GCP APIs (`workflows`, `monitoring`, `pubsub`, `bigquery`, `storage`) • Deklarative Cloud-Infrastruktur (`infra/main.tf`, `variables.tf`, `versions.tf`) • Serverless Cloud Workflow State-Machine (`infra/workflows/pipeline.yaml`) • REST Workflow-Runner (`src/cloud_workflow_runner.py`, `run_workflow.ps1`) • In-Workflow Quality Gate (Foreign-Key-Orphan-Check in BigQuery) • Grafana Cloud mit nativer Google Cloud Monitoring Data Source (Weg 1) • Dashboard-as-Code (`dashboards/youtube_dataops_monitoring.json`) • GitHub Actions CI/CD (`.github/workflows/ci.yml`, `release.yml`)	• **Lokaler Airflow Stack (`dags/`, `docker-compose-airflow.yml`, `start_airflow.ps1`, `stop_airflow.ps1`): VOLLSTÄNDIG ÜBERFLÜSSIG!** Zu 100% durch den serverlosen Google Cloud Workflow abgelöst. Spart 24/7 laufende Container und im Cloud-Betrieb ~300 €/Monat für Cloud Composer. • **Lokaler Prometheus & Grafana Container-Stack (`docker-compose-monitoring.yml`, `start_monitoring.ps1`, `stop_monitoring.ps1`): NICHT MEHR NÖTIG!** Grafana Cloud fragt GCP Monitoring direkt serverlos ab (0 lokale Daemons, 0 CPU-Last). • **Altes Spark Gold Script (`src/transform_gold.py`, `run_gold.ps1`):** Veraltet und redundant. • **Manuelle Klicks in der GCP Webkonsole ('ClickOps'):** Durch Terraform vollständig eliminiert.

Produktions-Leitlinie v2.1.0:
Wichtige Betriebs-Leitlinie für Release v2.1.0 (Aktuell): Für den regulären Betrieb und Tests müssen **weder Airflow (`start_airflow.ps1`) noch der lokale Monitoring-Stack (`start_monitoring.ps1`) gestartet werden!** Die gesamte Produktions-Pipeline läuft serverlos über Terraform (`run_infra.ps1`), BigQuery-Loader (`load_to_bigquery.ps1`), dbt Core (`run_dbt.ps1`), Cloud Workflows (`run_workflow.ps1`) und Grafana Cloud. Die alten v1.0-Dateien verbleiben ausschließlich zur Dokumentation des Migrationspfades im Repository.

3. End-to-End Systemarchitektur, Medallion Lakehouse & Repository-Struktur

Die Plattform folgt dem **Medallion Lakehouse Architektur-Muster** (Bronze -> Silver -> Gold), kombiniert dieses jedoch mit modernem In-Warehouse ELT (Extract-Load-Transform) und dimensionaler Star-Schema-Modellierung nach Ralph Kimball.

3.1 Die 6 Datenfluss-Schichten im Detail

1. Raw Landing & Streaming Ingestion (Kafka KRaft):

Der Streaming-Producer (`src/producer.py`) liest das kuratierte Sample `data/sample/us_videos_mini_500.csv` (326 Events) und simuliert das Echtzeit-Eintreffen von YouTube Trending Daten. 295 valide Datensätze fließen in das Kafka-Topic `topic_ingest`, während 31 synthetisch korrumpierte Datensätze (~10% Fehlerrate) direkt in das Dead-Letter-Queue-Topic `topic_err` geroutet werden. Zudem mutiert der Producer für das Ziel-Video `2kyS6SvSYSE` deterministisch Titel und Tags über drei Trending-Tage.

2. Bronze & Quarantäne Persistenz (Micro-Batch Consumers):

Der Bronze-Consumer (`src/consumer_bronze.py`) liest `topic_ingest`, reichert jeden Record mit UTC-Audit-Metadaten (`_ingested_at`, `_kafka_offset`, `_kafka_partition`) an und speichert Snappy-komprimierte Parquet-Dateien unter `data/bronze/`. Parallel isoliert `src/consumer_dlq.py` alle fehlerhaften Nachrichten aus `topic_err` inklusive `_quarantine_at` und `_error_reason` unter `data/quarantine/`.

3. Silver Layer: Apache Iceberg V2 & Broadcast-Transformation:

Ein ephemerer PySpark-Container (`de_spark`, `src/transform_silver.py`) liest die Bronze-Parquet-Dateien, konvertiert Datentypen (`views`, `likes`, `dislikes`, `comment_count` nach `long`; `trending_date` von `yy.dd.MM` nach `DATE`; `publish_time` nach `TIMESTAMP`) und führt einen Zero-Shuffle Broadcast-Join mit `data/raw/US_category_id.json` durch. Das Ergebnis wird im Dual-Write-Verfahren atomar in die **Apache Iceberg V2 Tabelle** `local.silver.videos` (`data/iceberg_warehouse/silver/videos/`) sowie als Parquet-Snapshot nach `data/silver/` geschrieben.

4. Cloud Ingestion & Google BigQuery Staging:

Das Skript `src/cloud_loader_bigquery.py` (`load_to_bigquery.ps1`) lädt die bereinigten Silver-Parquet-Daten idempotent (`WRITE_TRUNCATE`) in die Quelltabelle `silver_videos` im BigQuery-Dataset `youtube_analytics` (Location `EU`).

5. Gold Layer & Kimball Star-Schema (dbt Core):

dbt Core (`run_dbt.ps1`) übernimmt die In-Warehouse-Transformation in BigQuery: Über den Staging-View `stg_videos` und den SCD-2 Snapshot `dim_video_snapshot` entstehen die Dimensionstabellen `dim_video`, `dim_category` und `dim_date` sowie die Faktentabelle `fact_video_metrics` (mit Point-in-Time Join). Anschließend sichern 15 automatisierte dbt-Tests die Datenqualität ab.

6. Serverless Orchestrierung & Cloud Observability:

Der per Terraform bereitgestellte Google Cloud Workflow `youtube-dataops-pipeline` (`run_workflow.ps1`) aktualisiert `fact_video_metrics` serverlos in BigQuery und prüft über ein In-Workflow Quality Gate, dass exakt 0 verwaiste Fremdschlüssel (Orphan Records) existieren. Sämtliche Laufzeit-, Kosten- und Streaming-Metriken werden ohne lokale Daemons direkt in **Grafana Cloud** (`dashboards/youtube_dataops_monitoring.json`) visualisiert.

3.2 Vollständige Repository-Struktur mit Versions-Zuordnung

```
DE_Capstone_2026/
|-- .github/workflows/                    # [v2.1 Active] CI/CD Automation (GitHub Actions)
|   |-- ci.yml                            # [v2.1 Active] Syntax-, Flake8-, YAML- & JSON-Validierung
|   +-- release.yml                       # [v2.1 Active] Automatische GitHub Releases bei Tag-Push (v*)
|-- dashboards/                           # [v2.1 Active] Observability-as-Code
|   +-- youtube_dataops_monitoring.json   # [v2.1 Active] Grafana Cloud Dashboard (Plugin: stackdriver)
|-- docs/                                 # [v2.1 Active] Enterprise Dokumentation & Handbuch
|   +-- YouTube_DataOps_Platform_2026_Architektur_Repertorium.pdf
|-- dags/                                 # [v1.0 Legacy] Lokale Airflow DAGs (abgeloest durch Workflows)
|-- dbt/                                  # [v2.0 Active] Cloud DWH Transformationen (Kimball & SCD-2)
|   |-- dbt_project.yml & profiles.yml    # [v2.0 Active] dbt Konfiguration & BigQuery EU-Profil
|   |-- snapshots/dim_video_snapshot.sql  # [v2.0 Active] SCD-2 Snapshot (check_cols: title, tags, ...)
|   +-- models/
|       |-- staging/stg_videos.sql        # [v2.0 Active] Typisierung & Trimming View auf silver_videos
|       +-- marts/core/                   # [v2.0 Active] Kimball Star-Schema Marts & 15 Tests
|           |-- dim_video.sql             # [v2.0 Active] SCD-2 Dimension mit valid_from/to & is_current
|           |-- dim_category.sql          # [v2.0 Active] Conformed Kategorie-Dimension
|           |-- dim_date.sql              # [v2.0 Active] Kalender-Dimension (is_weekend, day_name)
|           |-- fact_video_metrics.sql    # [v2.0 Active] Fakten mit Point-in-Time SCD-2 Join
|           +-- schema.yml                # [v2.0 Active] 15 automatisierte dbt Quality Gate Tests
|-- infra/                                # [v2.1 Active] Infrastructure as Code (Terraform 1.17)
|   |-- versions.tf, variables.tf, main.tf# [v2.1 Active] 5 GCP Ressourcen (GCS, BQ, Pub/Sub, Workflows)
|   +-- workflows/pipeline.yaml           # [v2.1 Active] Serverless Cloud Workflow State-Machine
|-- docker/                               # Modulare Docker Compose Definitionen
|   |-- docker-compose-kafka.yml          # [v2.0 Active] Apache Kafka KRaft & Provectus Kafka-UI
|   |-- docker-compose-spark.yml          # [v2.0 Active] Ephemerer PySpark + Iceberg V2 Container
|   |-- docker-compose-airflow.yml        # [v1.0 Legacy] Alter Airflow 2.9 & Postgres Stack
|   +-- docker-compose-monitoring.yml     # [v1.0 Legacy] Alter lokaler Prometheus/Grafana Stack
|-- src/                                  # Python Quellcode
|   |-- producer.py                       # [v2.0 Active] Streaming Producer (10% DLQ + SCD-2 Mutation)
|   |-- consumer_bronze.py                # [v2.0 Active] Bronze Parquet Consumer (topic_ingest)
|   |-- consumer_dlq.py                   # [v2.0 Active] DLQ Quarantaene Consumer (topic_err)
|   |-- transform_silver.py               # [v2.0 Active] PySpark Broadcast-Join & Apache Iceberg V2
|   |-- transform_gold.py                 # [v1.0 Legacy] Altes Spark-Gold (abgeloest durch dbt)
|   |-- cloud_loader_bigquery.py          # [v2.0 Active] Idempotenter BigQuery Parquet Uploader
|   |-- cloud_workflow_runner.py          # [v2.1 Active] REST-Trigger & Live-Poller fuer Cloud Workflows
|   +-- generate_repertorium_pdf.py       # [v2.1 Active] ReportLab Generator fuer das PDF-Repertorium
+-- *.ps1 Runner-Skripte                  # [v2.1 Active] Kapselung aller CLI-Aufrufe fuer PowerShell
```

4. Streaming Ingestion & Resilienz (Kafka KRaft, DLQ & SCD-2 Mutation)

In produktiven DataOps-Architekturen ist unkontrollierter Datenverlust inakzeptabel. Gleichzeitig darf ein korrupter Datensatz niemals die gesamte Ingestion-Pipeline blockieren ('Poison Pill Problem'). Dieses Kapitel erläutert die Implementierung des Streaming-Layers mit Apache Kafka im KRaft-Modus, die 10%-Fault-Injection-Engine sowie die deterministische SCD-2-Titelmutation.

4.1 Warum Apache Kafka KRaft? (Eliminierung von ZooKeeper)

Historisch benötigte Apache Kafka einen separaten Apache ZooKeeper-Cluster zur Partition-Leader-Election und Metadaten-Verwaltung. Dies führte zu doppeltem Konfigurationsaufwand, Metadaten-Synchronisationslags und Split-Brain-Zuständen. Mit **Kafka Raft Metadata Mode (KRaft)** (`docker/docker-compose-kafka.yml`) wurde ZooKeeper vollständig eliminiert: Der Kafka-Broker agiert gleichzeitig als `broker,controller` und verwaltet den Cluster-State in einem internen, ereignisgesteuerten Raft-Quorum-Log. Alle Python-Clients verbinden sich explizit über `127.0.0.1:9092` (reines IPv4), um den unter Windows bekannten 2-Sekunden-IPv6-DNS-Fallback-Lag von `localhost` zu vermeiden.

4.2 Die Resilienz-Architektur: 10% Fault-Injection & Dead Letter Queue

Der Producer (`src/producer.py`) liest die 326 Datensätze aus `data/sample/us_videos_mini_500.csv` und implementiert eine präzise **Fault-Injection- und Quarantäne-Weiche**:

- **Stochastische 10% Fehlerrate (`ERROR_RATE = 0.10`):** Für alle Videos außer dem SCD-2-Zielvideo wird mit 10% Wahrscheinlichkeit (`random.random() < ERROR_RATE`) das numerische Feld `views` durch den ungültigen String `CORRUPT_INVALID_NUMBER` überschrieben und das Audit-Feld `_error_reason = 'Simulated fault injection: non-numeric views'` ergänzt.
- **Sofortiges DLQ-Routing (`topic_err`):** Korrupte Events werden nicht in den Hauptstrom eingespeist, sondern sofort an das Dead-Letter-Queue-Topic `topic_err` gesendet (im Referenzlauf: 31 Events).
- **Sauberer Hauptstrom (`topic_ingest`):** Alle validen Events fließen in `topic_ingest` (im Referenzlauf: 295 Events).
- **Entkoppelte Consumer mit Audit-Metadaten:** `src/consumer_bronze.py` (Gruppe `bronze_consumer_group`) konsumiert `topic_ingest`, ergänzt `_ingested_at` (UTC ISO-8601), `_kafka_offset` sowie `_kafka_partition` und schreibt Parquet nach `data/bronze/`. Analog isoliert `src/consumer_dlq.py` (Gruppe `dlq_consumer_group`) alle `topic_err`-Nachrichten mit `_quarantine_at` nach `data/quarantine/`. Beide Consumer nutzen `consumer_timeout_ms=5000` für saubere Micro-Batch-Terminierung.

4.3 Deterministische SCD-2 Titel- & Tag-Mutation im Producer

Auf YouTube ändern Creator häufig nach der Veröffentlichung den Titel oder die Tags eines Videos (A/B-Testing), um die Click-Through-Rate (CTR) zu steigern. Um die nachgelagerte SCD-2-Historisierung in BigQuery deterministisch zu beweisen, überwacht `src/producer.py` das Video mit der ID `2kyS6SvSYSE` und mutiert dessen Attribute über drei aufeinanderfolgende Erscheinungen im Stream:

- **Erscheinung 1 (Original aus CSV):** Titel: `WE WANT TO TALK ABOUT OUR MARRIAGE`
- **Erscheinung 2 (Mutation 1):** Titel: `WE WANT TO TALK ABOUT OUR MARRIAGE (Official Update)` | Tags: `marriage|relationship|update|vlog`
- **Erscheinung 3 (Mutation 2):** Titel: `WE WANT TO TALK ABOUT OUR MARRIAGE [THE TRUTH]` | Tags: `marriage|relationship|update|vlog|truth`

Original-Codeausschnitt aus `src/producer.py`:

```
# src/producer.py (Original-Auszug: SCD-2 Mutation & 10% DLQ Fault-Injection)
SCD2_TARGET_VIDEO_ID = "2kyS6SvSYSE"
SCD2_MUTATIONS = {
    2: {
        "title": "WE WANT TO TALK ABOUT OUR MARRIAGE (Official Update)",
        "tags": "marriage|relationship|update|vlog"
    },
    3: {
        "title": "WE WANT TO TALK ABOUT OUR MARRIAGE [THE TRUTH]",
        "tags": "marriage|relationship|update|vlog|truth"
    }
}

# 1. Deterministische SCD-2 Mutation fuer unser Ziel-Video
if vid == SCD2_TARGET_VIDEO_ID:
    scd2_occurrence_count += 1
    if scd2_occurrence_count in SCD2_MUTATIONS:
        row.update(SCD2_MUTATIONS[scd2_occurrence_count])

# 2. Fault Injection (10% korrupte Daten -> Dead Letter Queue)
if vid != SCD2_TARGET_VIDEO_ID and random.random() < ERROR_RATE:
    corrupted_row = dict(row)
    corrupted_row["views"] = "CORRUPT_INVALID_NUMBER"
    corrupted_row["_error_reason"] = "Simulated fault injection: non-numeric views"
    producer.send(TOPIC_ERR, key=vid, value=corrupted_row)
    err_count += 1
else:
    producer.send(TOPIC_INGEST, key=vid, value=row)
    valid_count += 1
```

5. Ephemerer Compute & Open Table Format (Apache Iceberg V2)

Im Zeitalter moderner Cloud Lakehouses reichen lose Parquet-Dateien im Object Store für produktive Workloads nicht mehr aus. Unternehmen benötigen ACID-Transaktionen, Snapshot-Isolation, Schema-Evolution und Time-Travel. Dieses Kapitel beschreibt die Implementierung des Silver-Layers mit PySpark 3.5 und **Apache Iceberg V2** (`src/transform_silver.py`).

5.1 Warum lose Parquet-Dateien in Produktion scheitern

Parquet ist ein exzellentes spaltenorientiertes Dateiformat — aber ein Ordner voller Parquet-Dateien ist **keine Datenbanktabelle**:

- **Fehlende Atomarität (Dirty Reads):** Bricht ein Spark-Schreibjob nach 40 von 50 Parquet-Dateien ab, bleibt ein halb geschriebenes, korruptes Verzeichnis zurück.
- **Teures Directory Listing `O(N)`:** Abfrage-Engines müssen bei losen Dateien langsame Dateisystem-Listings (`listStatus`) über alle Unterordner ausführen.
- **Kein Time-Travel & kein sicheres Schema-Update:** Überschriebene Parquet-Dateien sind unwiderruflich verloren; Spaltenumbenennungen brechen bestehende Leser.

5.2 Die Apache Iceberg V2 Metadaten-Hierarchie (`local.silver.videos`)

Metadaten-Ebene	Physischer Pfad im Projekt	Funktion & ACID-Mechanismus
Iceberg Catalog	`spark.sql.catalog.local (type = hadoop)`	Verwaltet den atomaren Zeiger auf die aktuelle `vN.metadata.json` im Warehouse `/app/data/iceberg_warehouse`.
Table Metadata	`silver/videos/metadata/ v1.metadata.json`	Speichert das Tabellen-Schema (Field-IDs), Partitionsspezifikationen und die vollständige Snapshot-Historie.
Manifest List	`silver/videos/metadata/ snap-*.avro`	Verzeichnet alle Manifest-Dateien eines einzelnen Commits inkl. Partitions-Statistiken für schnelles Pruning.
Manifest Files	`silver/videos/metadata/ *-m0.avro`	Trackt jede einzelne Parquet-Datendatei mit Zeilenanzahl und Spalten-Min/Max-Werten (Data Skipping).
Data Files	`silver/videos/data/ *.parquet`	Die Snappy-komprimierten, typisierten und per Broadcast-Join angereicherten YouTube-Events.

5.3 Ephemerer Spark Compute, Broadcast-Join & Dual-Write

Über `run_silver.ps1` wird der Container `de_spark` (`docker/docker-compose-spark.yml`) on-demand gestartet, führt `src/transform_silver.py` aus und beendet sich danach sofort wieder (0 € Idle-Ressourcen). Dabei liest Spark die Kategorie-Referenz `data/raw/US_category_id.json` und erzwingt über `broadcast(df_categories)` einen **Zero-Shuffle Broadcast-Join** im Arbeitsspeicher. Anschließend schreibt der Job sowohl atomar in die Iceberg-Tabelle `local.silver.videos` als auch nach `/app/data/silver`, damit `src/cloud_loader_bigquery.py` die Daten nahtlos nach BigQuery hochladen kann:

```
# src/transform_silver.py (Original-Auszug: Iceberg Session, Broadcast Join & Commit)
def create_spark_session():
    return (
        SparkSession.builder.appName("DE_Capstone_Silver_Iceberg").master("local[*]")
        .config("spark.sql.extensions", "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions"
)
        .config("spark.sql.catalog.local", "org.apache.iceberg.spark.SparkCatalog")
```

```
        .config("spark.sql.catalog.local.type", "hadoop")
        .config("spark.sql.catalog.local.warehouse", "/app/data/iceberg_warehouse")
        .getOrCreate()
    )

# Typisierung & Datums-Harmonisierung ('17.14.11' -> yy.dd.MM)
df_cleaned = (
    df_bronze.withColumn("category_id", col("category_id").cast("string"))
    .withColumn("views", col("views").cast("long"))
    .withColumn("likes", col("likes").cast("long"))
    .withColumn("dislikes", col("dislikes").cast("long"))
    .withColumn("comment_count", col("comment_count").cast("long"))
    .withColumn("publish_time", to_timestamp(col("publish_time")))
    .withColumn("trending_date", to_date(col("trending_date"), "yy.dd.MM"))
)

# Zero-Shuffle Broadcast Join & atomarer Apache Iceberg V2 Commit
df_silver = df_cleaned.join(broadcast(df_categories), on="category_id", how="left").na.fill({"category_name
": "Unknown"})
df_silver.writeTo("local.silver.videos").createOrReplace()
spark.read.table("local.silver.videos.history").show(truncate=False)
```

6. Cloud Data Warehouse & Kimball Star-Schema (dbt Core & SCD-2)

Nachdem `src/cloud_loader_bigquery.py` (`load_to_bigquery.ps1`) die 295 bereinigten Silver-Events per `WRITE_TRUNCATE` als Quelltabelle `silver_videos` in das BigQuery-Dataset `youtube_analytics` (Location EU) geladen hat, übernimmt **dbt Core 1.13** (`run_dbt.ps1`) die In-Warehouse-Transformation (ELT) in ein dimensionales **Star-Schema nach Ralph Kimball**.

6.1 Die 6 dbt-Artefakte im BigQuery Dataset `youtube_analytics`

Modell / Tabelle	Materialisierung	Primär-/Surrogate-Key	Pfad & Transformations-Logik
stg_videos	View	`video_id` + `trending_date`	`models/staging/stg_videos.sql`: Trimming von Strings, Casting auf `INT64`, `DATE` und `TIMESTAMP`, Filter `video_id is not null`.
dim_video_snapshot	SCD-2 Snapshot Table	`dbt_scd_id` (MD5)	`snapshots/dim_video_snapshot.sql`: Überwacht `['title', 'channel_title', 'category_id', 'tags']` via `strategy='check'`.
dim_video	Table (Dimension)	`video_key` (`dbt_scd_id`)	`models/marts/core/dim_video.sql`: Exponiert `video_key`, `valid_from`, `valid_to` und berechnet das Flag `is_current`.
dim_category	Table (Conformed Dim)	`category_key` (`INT64`)	`models/marts/core/dim_category.sql`: Deduplizierte Kategorie-Dimension (`category_key`, `category_id`, `category_name`).
dim_date	Table (Date Spine)	`date_key` (`YYYYMMDD`)	`models/marts/core/dim_date.sql`: Berechnet `year`, `month`, `month_name`, `day`, `day_of_week`, `day_name` und `is_weekend`.
fact_video_metrics	Table (Fact Table)	`metric_key` (MD5 Hex)	`models/marts/core/fact_video_metrics.sql`: Tägliche Kennzahlen, `interaction_ratio` und Point-in-Time Join auf `dim_video`.

6.2 Deep Dive: SCD-2 Snapshot (`dim_video_snapshot.sql`) & Deduplizierung

Da ein Video im Streaming-Batch mehrfach vorkommen kann, dedupliziert der Snapshot die Quelle mittels `qualify row_number() over (partition by video_id order by trending_date desc, _ingested_at desc) = 1` und prüft auf Änderungen in den überwachten Attributen:

```
-- dbt/snapshots/dim_video_snapshot.sql (Original-Code)
{% snapshot dim_video_snapshot %}
{{
    config(
      target_schema='youtube_analytics',
      strategy='check',
      unique_key='video_id',
      check_cols=['title', 'channel_title', 'category_id', 'tags'],
      invalidate_hard_deletes=True
    )
}}
select video_id, title, channel_title, category_id, tags
from {{ ref('stg_videos') }}
qualify row_number() over (partition by video_id order by trending_date desc, _ingested_at desc) = 1
{% endsnapshot %}
```

6.3 Point-in-Time Join in der Faktentabelle (`fact_video_metrics.sql`)

Beim Aufbau der Faktentabelle wird jedes Trending-Event exakt mit derjenigen Version von `dim_video` verknüpft, die zum jeweiligen `trending_date` gültig war (`valid_from` bis `valid_to`). Zudem verhindert `safe_divide` Division-by-Zero-Fehler bei der Berechnung der `interaction_ratio`:

```
-- dbt/models/marts/core/fact_video_metrics.sql (Original-Code)
{{ config(materialized='table') }}

with source_data as ( select * from {{ ref('stg_videos') }} ),
dim_v as ( select * from {{ ref('dim_video') }} )

select
    to_hex(md5(concat(s.video_id, '_', cast(s.trending_date as string)))) as metric_key,
    dim_v.video_key,
    cast(s.category_id as int64) as category_key,
    cast(format_date('%Y%m%d', s.trending_date) as int64) as date_key,
    s.views, s.likes, s.dislikes, s.comment_count,
    safe_divide(s.likes + s.dislikes + s.comment_count, s.views) as interaction_ratio,
    s._ingested_at
from source_data s
inner join dim_v
    on s.video_id = dim_v.video_id
    and s.trending_date >= cast(dim_v.valid_from as date)
    and (dim_v.valid_to is null or s.trending_date < cast(dim_v.valid_to as date))
```

7. DataOps Quality Gates: 15 dbt Tests & In-Workflow Validierung

Gemäß dem DataOps-Prinzip **'Shift Left'** verlagert die Plattform Qualitätsprüfungen direkt in den Transformations- und Orchestrierungszyklus. Da Cloud Data Warehouses wie BigQuery standardmäßig keine Primary-Key- oder Foreign-Key-Constraints beim Schreiben erzwingen, implementiert das Projekt ein zweistufiges Schutzschild aus **15 deklarativen dbt Data Tests** (`dbt/models/marts/core/schema.yml`) und einem **serverlosen In-Workflow Quality Gate** (`infra/workflows/pipeline.yaml`).

7.1 Exakte Aufschlüsselung aller 15 dbt Data Tests (`schema.yml`)

Mart-Tabelle	Spalte	Definierte dbt-Tests	Anzahl	Abgesicherte Invariante
dim_video	`video_key` `video_id`	`unique`, `not_null` `not_null`	**3 Tests**	Jede SCD-2-Versionszeile hat einen eindeutigen Surrogate-Hash (`dbt_scd_id`) und einen Natural Key.
dim_category	`category_key`	`unique`, `not_null`	**2 Tests**	Keine doppelten oder leeren Kategorienschlüssel in der Conformed Dimension.
dim_date	`date_key`	`unique`, `not_null`	**2 Tests**	Jeder Kalendertag im Format `YYYYMMDD` existiert exakt einmal.
fact_video_metrics	`metric_key`	`unique`, `not_null`	**2 Tests**	Verhindert doppelte Faktenzeilen für dasselbe Video am selben Trending-Tag.
fact_video_metrics	`video_key`	`not_null` + `relationships` -> `dim_video`	**2 Tests**	**Referenzielle Integrität (SCD-2):** Jeder Fakt zeigt auf eine existierende `dim_video`-Version.
fact_video_metrics	`category_key`	`not_null` + `relationships` -> `dim_category`	**2 Tests**	**Referenzielle Integrität:** Jede Faktenzeile besitzt eine valide Kategorie.
fact_video_metrics	`date_key`	`not_null` + `relationships` -> `dim_date`	**2 Tests**	**Referenzielle Integrität:** Jede Faktenzeile besitzt einen gültigen Kalendertag.
SUMME	**Alle 4 Core Marts**	**4x unique, 8x not_null, 3x FK**	**15 Tests**	**Ergebnis im Produktivlauf:** `PASS=15 WARN=0 ERROR=0 SKIP=0`

7.2 Das serverlose In-Workflow Quality Gate (`pipeline.yaml`)

Zusätzlich zu `run_dbt.ps1 test` führt der Google Cloud Workflow bei jedem Lauf in der Cloud automatisch eine Foreign-Key-Orphan-Prüfung in BigQuery aus. Werden verwaiste Schlüssel (`invalid_count > 0`) entdeckt, wirft der Workflow im Schritt `evaluate_quality_gate` sofort eine Exception (`raise`) und verhindert eine Freigabe fehlerhafter Daten.

Formeller Qualitätsnachweis:
Verifiziertes Testergebnis (`run_dbt.ps1 test` & `run_workflow.ps1`):
• dbt Core: `Done. PASS=15 WARN=0 ERROR=0 SKIP=0 TOTAL=15`
• Cloud Workflow: `{"message":"DataOps Pipeline completed successfully. 0 quality errors detected.","status":"SUCCESS"}`

8. Infrastructure as Code (Terraform 1.17 Cloud Provisioning)

Manuelles Zusammenklicken von Cloud-Ressourcen ('ClickOps') führt unweigerlich zu Konfigurations-Drift und nicht reproduzierbaren Umgebungen. In Release `v2.1.0` wird die gesamte Google Cloud Infrastruktur über **Terraform 1.17** (`infra/main.tf`, `infra/variables.tf`, `infra/versions.tf`) deklarativ verwaltet und über den PowerShell-Wrapper `run_infra.ps1` gesteuert.

8.1 Die 5 bereitgestellten Google Cloud Ressourcen im Detail

Terraform Ressource	Live GCP Name / ID	Konfiguration & FinOps-Schutz
`google_storage_bucket.lakehouse_storage`	`lakehouse-youtube-cloudanalytics-514018`	Location `EU`, `uniform_bucket_level_access = true`, `versioning { enabled = true }`, `force_destroy = true`.
`google_bigquery_dataset.youtube_analytics`	`youtube_analytics`	Location `EU`, `delete_contents_on_destroy = true` (garantiert rückstandsloses Löschen bei `destroy`).
`google_pubsub_topic.youtube_ingest`	`topic-youtube-ingest`	Serverloses Cloud-Pub/Sub-Topic für den regulären Event-Ingest-Strom.
`google_pubsub_topic.youtube_dlq`	`topic-youtube-dlq`	Serverloses Cloud-Pub/Sub-Topic für isolierte Dead-Letter-Queue-Nachrichten.
`google_workflows_workflow.youtube_dataops_pipeline`	`youtube-dataops-pipeline`	Region `europe-west3` (Frankfurt), nutzt `var.service_account_email` und injiziert `workflows/pipeline.yaml`.

8.2 Vollständiger Code von `infra/main.tf`

```
# infra/main.tf (Vollstaendiger Original-Code)
provider "google" {
  project     = var.project_id
  region      = var.region
  credentials = file(var.credentials_file)
}

resource "google_storage_bucket" "lakehouse_storage" {
  name                        = "lakehouse-youtube-${var.project_id}"
  location                    = var.location
  force_destroy               = true
  uniform_bucket_level_access = true
  versioning { enabled = true }
}

resource "google_bigquery_dataset" "youtube_analytics" {
  dataset_id                  = "youtube_analytics"
  friendly_name               = "YouTube Video Analytics DWH"
  location                    = var.location
  delete_contents_on_destroy = true
}

resource "google_pubsub_topic" "youtube_ingest" {
  name = "topic-youtube-ingest"
}

resource "google_pubsub_topic" "youtube_dlq" {
  name = "topic-youtube-dlq"
}

resource "google_workflows_workflow" "youtube_dataops_pipeline" {
  name            = "youtube-dataops-pipeline"
  region          = var.region
  description     = "Serverlose Orchestrierung fuer YouTube DWH Star-Schema & Quality Gates"
  service_account = var.service_account_email
  source_contents = file("${path.module}/workflows/pipeline.yaml")
}
```

9. Serverlose Orchestrierung (Google Cloud Workflows State-Machine)

In Release `v2.1.0` löst **Google Cloud Workflows** den lokalen Airflow-Container-Stack aus `v1.0.0` vollständig ab. Während ein gemanagter Airflow-Cluster (Cloud Composer) durch dauerhaft laufende GKE-Knoten rund 300 € bis 450 € pro Monat kostet, arbeitet Cloud Workflows zu 100% serverlos mit **0,00 € Idle-Kosten** (5.000 Schritte/Monat im Always Free Tier inklusive).

9.1 Architektur & Parser-Härtung von `infra/workflows/pipeline.yaml`

Die State-Machine nutzt den offiziellen Google Cloud BigQuery Connector (`googleapis.bigquery.v2.jobs.query`) und berücksichtigt zwei wichtige Produktions-Restriktionen der Cloud Workflows Engine:

- **400-Zeichen-Limit für Ausdrücke (`${...}`):** Da Cloud Workflows dynamische Ausdrücke auf maximal 400 Zeichen begrenzt, ist das SQL-Statement in `prepare_fact_query` sauber in vier Teilvariablen (`q_target`, `q_select`, `q_from`, `q_join`) modularisiert und wird über `${q_target + q_select + q_from + q_join}` konkateniert.
- **Strikte Trennung von `assign` und `switch`:** In Cloud Workflows darf ein Schritt nicht gleichzeitig `assign` und `switch` enthalten. Deshalb liest `get_quality_result` zunächst `invalid_count` aus, bevor `evaluate_quality_gate` die Bedingung `${invalid_count > 0}` prüft.

9.2 Vollständiger Code von `infra/workflows/pipeline.yaml`

```
# infra/workflows/pipeline.yaml (Vollstaendiger Original-Code)
main:
  params: [args]
  steps:
    - init_constants:
        assign:
          - project_id: "cloudanalytics-514018"
          - dataset_id: "youtube_analytics"
    - check_silver_data:
        call: googleapis.bigquery.v2.jobs.query
        args:
          projectId: ${project_id}
          body:
            query: ${"SELECT count(*) as total_records FROM `" + project_id + "." + dataset_id + ".silver_v
ideos`"}
            useLegacySql: false
        result: silver_check_result
    - prepare_fact_query:
        assign:
          - q_target: ${"CREATE OR REPLACE TABLE `" + project_id + "." + dataset_id + ".fact_video_metrics`
 AS "}
          - q_select: "SELECT TO_HEX(MD5(CONCAT(s.video_id, '_', CAST(s.trending_date AS STRING)))) AS metr
ic_key, dim_v.video_key, CAST(s.category_id AS INT64) AS category_key, CAST(FORMAT_DATE('%Y%m%d', s.trendin
g_date) AS INT64) AS date_key, s.views, s.likes, s.dislikes, s.comment_count, SAFE_DIVIDE(s.likes + s.disli
kes + s.comment_count, s.views) AS interaction_ratio, s._ingested_at "
          - q_from: ${"FROM `" + project_id + "." + dataset_id + ".stg_videos` s INNER JOIN `" + project_id
 + "." + dataset_id + ".dim_video` dim_v "}
          - q_join: "ON s.video_id = dim_v.video_id AND s.trending_date >= CAST(dim_v.valid_from AS DATE) A
ND (dim_v.valid_to IS NULL OR s.trending_date < CAST(dim_v.valid_to AS DATE))"
          - fact_query: ${q_target + q_select + q_from + q_join}
    - refresh_fact_table:
        call: googleapis.bigquery.v2.jobs.query
        args:
          projectId: ${project_id}
          body:
            query: ${fact_query}
            useLegacySql: false
        result: fact_refresh_result
    - quality_gate_validation:
        call: googleapis.bigquery.v2.jobs.query
        args:
          projectId: ${project_id}
          body:
            query: ${"SELECT count(*) as invalid_keys FROM `" + project_id + "." + dataset_id + ".fact_vide
o_metrics` f LEFT JOIN `" + project_id + "." + dataset_id + ".dim_video` v ON f.video_key = v.video_key WHE
RE v.video_key IS NULL"}
            useLegacySql: false
        result: quality_result
    - get_quality_result:
        assign:
          - invalid_count: ${int(quality_result.rows[0].f[0].v)}
    - evaluate_quality_gate:
        switch:
          - condition: ${invalid_count > 0}
            raise: ${"Quality Gate FAILED! Found " + string(invalid_count) + " orphan records in fact_video
_metrics."}
    - finish_success:
        return:
          status: "SUCCESS"
          message: "DataOps Pipeline completed successfully. 0 quality errors detected."
```

10. Cloud-Native Observability (Grafana Cloud & GCP Monitoring - Weg 1)

In Release `v2.1.0` wurde das Monitoring auf **'Weg 1: Cloud-Native Observability'** umgestellt. Anstatt lokale Prometheus- und Grafana-Container auf dem Entwickler-PC laufen zu lassen, verbindet sich **Grafana Cloud** über das offizielle **Google Cloud Monitoring** Plugin (`pluginId: stackdriver`) direkt mit den Telemetrie-APIs der Google Cloud Platform — mit **0 lokalen Daemons und 0 CPU/RAM-Belastung**.

10.1 Die 8 Panels des Grafana Dashboards (`youtube_dataops_monitoring.json`)

Row / Panel-Titel	Visualisierung	GCP Metric Type (`stackdriver`)	Operativer & FinOps Nutzen
1. Workflow Executions *(Row: High-Level KPIs)*	Stat Panel (Grün)	`workflows.googleapis.com/finished_execution_count`	Gesamtzahl abgeschlossener Cloud-Workflow-Läufe (`REDUCE_SUM`, `ALIGN_DELTA`).
2. BigQuery Queries *(Row: High-Level KPIs)*	Stat Panel (Blau)	`bigquery.googleapis.com/query/count`	Überwacht die DWH-Abfragefrequenz durch dbt und Cloud Workflows.
3. Pub/Sub Activity *(Row: High-Level KPIs)*	Stat Panel (Violett)	`pubsub.googleapis.com/topic/send_message_operation_count`	Gesamtes Streaming-Nachrichtenvolumen über alle Cloud-Topics.
4. BigQuery Bytes Billed *(Row: High-Level KPIs)*	Stat Panel (Ampel)	`bigquery.googleapis.com/query/scanned_bytes_billed`	**FinOps-Wächter:** Grün < 100 MB, Orange ab 100 MB, Rot ab 1 GB gescannten Daten.
5. Workflow Executions by Status *(Row: Orchestration)*	Stacked Bar Chart	`workflows.googleapis.com/finished_execution_count`	Gruppiert nach `metric.label.status` (`SUCCEEDED` vs. `FAILED`) über die Zeit.
6. Workflow Execution Duration (p99) *(Row: Orchestration)*	Time Series Line (s)	`workflows.googleapis.com/execution_durations`	Misst die 99.-Perzentil-Latenz (`ALIGN_PERCENTILE_99`) der Pipeline-Durchläufe.
7. Pub/Sub Topics (Ingest vs DLQ) *(Row: DWH & DLQ Health)*	Stacked Bar Chart	`pubsub.googleapis.com/topic/send_message_operation_count`	Gruppiert nach `resource.label.topic_id` (`topic-youtube-ingest` vs. `topic-youtube-dlq`).
8. BigQuery Data Scanned Volume *(Row: DWH & DLQ Health)*	Time Series Area	`bigquery.googleapis.com/query/scanned_bytes_billed`	Historischer Verlauf des Abfrage-Scanvolumens zur Erkennung teurer Full-Table-Scans.

Grafana Schema-Kompatibilität:
Technischer Hinweis zum Grafana-Import (`pluginId: stackdriver`): Intern identifiziert Grafana 10+ die Datenquelle 'Google Cloud Monitoring' unter der historischen Plugin-ID `stackdriver`. In `dashboards/youtube_dataops_monitoring.json` ist diese ID exakt hinterlegt, sodass beim JSON-Import sofort das Dropdown zur Auswahl der verbundenen GCP-Datenquelle erscheint.

11. DataOps CI/CD Pipelines, GitOps & Release-Management

Im Data Engineering trifft deklarativer Code auf persistenten Daten-State. Dieses Kapitel zeigt sowohl die im Repository aktiven **GitHub Actions Workflows** (`.github/workflows/ci.yml` und `release.yml`) als auch das **Enterprise-Zielbild für Multi-Environment Promotion (Dev -> Staging -> Prod)**.

11.1 Die aktiven GitHub Actions Workflows im Repository

```
# .github/workflows/ci.yml (Original-Code)
name: DataOps CI & Validation
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  validate-pipeline:
    name: Code Quality & Schema Verification
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
      - name: Set up Python 3.10
        uses: actions/setup-python@v5
        with:
          python-version: "3.10"
          cache: "pip"
      - name: Install Python Dependencies
        run: |
          python -m pip install --upgrade pip
          pip install flake8 pyyaml
      - name: Python Syntax & Lint Check
        run: |
          python -m compileall src/
          flake8 src/ --count --select=E9,F63,F7,F82 --show-source --statistics
      - name: Validate Cloud Workflows YAML
        run: |
          python -c "import yaml; yaml.safe_load(open('infra/workflows/pipeline.yaml'))"
      - name: Validate Grafana Dashboard JSON
        run: |
          python -c "import json; json.load(open('dashboards/youtube_dataops_monitoring.json'))"
```

Ergänzend triggert `.github/workflows/release.yml` bei jedem Push eines Versions-Tags (`v*`, wie `v1.0.0`, `v2.0.0`, `v2.1.0`) die Action `softprops/action-gh-release@v2` und publiziert automatisch ein GitHub Release inklusive generierter Release Notes.

11.2 Enterprise-Praxis: Drei-Zonen-Modell, Slim CI & Keyless OIDC

- **Umgebungs-Isolation (Dev / Staging / Prod):** Entwickler arbeiten lokal gegen ein isoliertes Dataset (`youtube_analytics_dev`). Pull Requests bauen in einer ephemeren Staging-Umgebung (`youtube_analytics_ci_<PR_ID>`). Auf `youtube_analytics_prod` hat kein Mensch Schreibrechte — ausschließlich der CI/CD-Service-Account.
- **dbt Slim CI (`state:modified+`):** In großen DWHs baut die CI-Pipeline bei einem Pull Request nicht alle 500 Modelle neu, sondern führt via `dbt build --select state:modified+ --defer --state ./prod-manifest` nur geänderte Modelle und deren direkte Kinder aus.
- **Keyless Authentication via Workload Identity Federation (OIDC):** Statt statischer JSON-Keys in GitHub Secrets tauscht GitHub Actions zur Laufzeit ein kurzlebiges OIDC-JWT-Token gegen ein 1 Stunde gültiges

GCP OAuth2-Token (`google-github-actions/auth@v2`).

- **Remote Terraform State Locking:** Der Terraform-State (`terraform.tfstate`) liegt im Enterprise-Betrieb in einem versionierten GCS-Bucket mit automatischem State-Locking, damit niemals zwei Pipelines gleichzeitig `terraform apply` ausführen.

12. Architectural Decision Records (ADRs 001 bis 006) im Detail

Architectural Decision Records (ADRs) dokumentieren fundamentale Architekturentscheidungen nachvollziehbar mit Kontext, verworfenen Alternativen, Trade-offs und Begründungen.

ADR-001: Apache Kafka KRaft vs. ZooKeeper

Entscheidung: Einsatz von Apache Kafka im ZooKeeper-losen KRaft-Modus (`docker-compose-kafka.yml`).
Kontext: Das 2021er Setup litt unter ZooKeeper-Timeouts und Split-Brain-Problemen auf Windows-Hosts.
Alternativen: Kafka mit ZooKeeper, RabbitMQ, AWS Kinesis.
Begründung & Trade-offs: KRaft halbiert die Container-Anzahl, beschleunigt Metadaten-Operationen um den Faktor 10 und eliminiert ZooKeeper als Fehlerquelle.

ADR-002: Apache Iceberg V2 Table Format vs. Pure Parquet

Entscheidung: Nutzung von Apache Iceberg V2 (`local.silver.videos`) für den Silver Lakehouse Layer.
Kontext: Reine Parquet-Ordner bieten keine ACID-Garantien, keine Snapshot-Isolation und erfordern teure `O(N)` Directory-Listings.
Alternativen: Delta Lake, Apache Hudi, lose Parquet-Dateien.
Begründung & Trade-offs: Iceberg V2 trennt physische Parquet-Daten strikt von logischen Metadaten (`v1.metadata.json`, `snap-*.avro`), erlaubt atomare Commits, Time-Travel und volle Engine-Neutralität (Spark, BigQuery BigLake, Trino).

ADR-003: Ephemerer Docker PySpark vs. Permanenter Spark-Cluster

Entscheidung: Ausführung von PySpark-Jobs in einem ephemeren On-Demand-Container (`de_spark`).
Kontext: Ein dauerhaft laufender Spark-Cluster (Standalone/Dataproc) verbraucht rund um die Uhr RAM/CPU.
Alternativen: GCP Dataproc Cluster, Databricks All-Purpose Cluster, lokales PySpark auf Host.
Begründung & Trade-offs: Ephemerer Compute erzeugt 0 € Leerlaufkosten, kapselt Java/Hadoop/Iceberg-JARs sauber vom Windows-Host ab und nutzt über `broadcast()` einen Zero-Shuffle Join.

ADR-004: Kimball Star-Schema & SCD-2 via dbt Core vs. One Big Table (OBT)

Entscheidung: Modellierung als relationales Kimball Star-Schema mit SCD-2 Snapshot (`dim_video_snapshot`) in BigQuery.
Kontext: Flache Tabellen (OBT) überschreiben bei Titeländerungen die Historie (SCD-1) und duplizieren Dimensionsattribute.
Alternativen: Flaches Spark-Gold-Skript (`transform_gold.py` aus v1.0), Data Vault 2.0.
Begründung & Trade-offs: Das Star-Schema trennt Fakten (`fact_video_metrics`) von Dimensionen (`dim_video`, `dim_category`, `dim_date`), ermöglicht exakte Point-in-Time Joins und wird durch 15 dbt-Tests abgesichert.

ADR-005: Google Cloud Workflows vs. Apache Airflow / Cloud Composer

Entscheidung: Ablösung von Apache Airflow (v1.0) durch serverlose Google Cloud Workflows (v2.1).
Kontext: Cloud Composer (Managed Airflow auf GCP) kostet mindestens ~300 €/Monat, selbst im Leerlauf.
Alternativen: Lokaler Airflow Docker-Stack, Cloud Composer, Prefect.
Begründung & Trade-offs: Cloud Workflows startet in unter 200 ms via REST, orchestriert BigQuery nativ, prüft das Foreign-Key Quality Gate in der Cloud und kostet im Always Free Tier exakt 0,00 €.

ADR-006: Grafana Cloud Direct Monitoring (Weg 1) vs. Lokale Prometheus-Container

Entscheidung: Direkte Anbindung von Grafana Cloud an Google Cloud Monitoring (`pluginId: stackdriver`).
Kontext: Lokale Prometheus/Grafana-Container (`docker-compose-monitoring.yml`) belegen lokal RAM/Ports und kennen keine nativen GCP-Metriken.
Alternativen: Lokaler Prometheus mit GCP-Exporter, Google Cloud Console Charts.
Begründung & Trade-offs: 0 lokale Daemons, direkte Abfrage von `workflows`, `bigquery` und `pubsub` Metriken über den Service Account (`roles/monitoring.viewer`) und versioniertes Dashboard-as-Code.

13. Deppensicheres Betriebshandbuch (9-Schritte-Anleitung & Playbooks)

Dieses Kapitel enthält die vollständige, **deppensichere Schritt-für-Schritt-Anleitung (End-to-End)** inklusive aller Voraussetzungen, IAM-Rollen, GCP-APIs, PowerShell-Runner und Fehlerbehebungs-Playbooks.

13.1 Checkliste der Voraussetzungen (IAM-Rollen & GCP APIs)

- **Lokale Systemumgebung:** Windows 10/11 mit PowerShell, Docker Desktop (gestartet, WSL2-Backend), Python 3.10+ sowie Terraform 1.17.
- **GCP Projekt & Service Account Key:** Projekt-ID `cloudanalytics-514018` (Region `europe-west3` / Location EU). Der JSON-Key des Service Accounts `cloudanalytics-514018@cloudanalytics-514018.iam.gserviceaccount.com` liegt als `gcp_service_account.json` im Projekt-Root (via `.gitignore` geschützt).
- **Die 6 zwingend erforderlichen GCP IAM-Rollen auf dem Service Account:**
  1. `roles/bigquery.admin` (BigQuery Administrator)
  2. `roles/storage.admin` (Storage Administrator)
  3. `roles/pubsub.admin` (Pub/Sub Administrator)
  4. `roles/workflows.admin` (Workflows Administrator)
  5. `roles/iam.serviceAccountUser` (Dienstkonto-Nutzer — zwingend für Workflow-Deployment!)
  6. `roles/monitoring.viewer` (Monitoring-Betrachter — zwingend für Grafana Cloud!)
- **Die 5 zwingend aktivierten Google Cloud APIs:**
  `workflows.googleapis.com`, `monitoring.googleapis.com`, `pubsub.googleapis.com`, `bigquery.googleapis.com`, `storage.googleapis.com`.

13.2 Die 9-Schritte End-to-End Ausführungssequenz (PowerShell)

```
# SCHRITT 1: Python Virtual Environment aktivieren & Abhaengigkeiten installieren
cd D:\Projekte\CloudEng\DE_Capstone_2026
python -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt

# SCHRITT 2: Cloud-Infrastruktur (5 GCP Ressourcen) per Terraform ausrollen
.\run_infra.ps1 init
.\run_infra.ps1 plan
.\run_infra.ps1 apply     # -> Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

# SCHRITT 3: Streaming Ingestion (Kafka KRaft, 10% DLQ & SCD-2 Mutation)
.\start_kafka.ps1         # Startet Kafka Broker (:9092) & Kafka-UI (http://localhost:8080)
.\run_producer.ps1        # Sendet 295 Clean Events (topic_ingest) + 31 DLQ Events (topic_err)
.\run_bronze.ps1          # Schreibt 295 Events als Parquet nach data/bronze/
.\run_dlq.ps1             # Isoliert 31 korrupte Events als Parquet nach data/quarantine/

# SCHRITT 4: Silver Lakehouse Transformation (Ephemerer PySpark & Apache Iceberg V2)
.\run_silver.ps1          # Broadcast-Join + Commit in Iceberg (local.silver.videos) & data/silver/

# SCHRITT 5: Bereinigte Silver-Daten idempotent nach Google BigQuery laden
.\load_to_bigquery.ps1    # Laedt 295 Zeilen in cloudanalytics-514018.youtube_analytics.silver_videos

# SCHRITT 6: dbt Kimball Star-Schema, SCD-2 Snapshot & 15 Quality-Gate-Tests ausfuehren
.\run_dbt.ps1 debug       # Prueft BigQuery-Verbindung
.\run_dbt.ps1 snapshot    # Historisiert Titel-Mutationen (2kyS6SvSYSE) in dim_video_snapshot
.\run_dbt.ps1 run         # Baut stg_videos, dim_video, dim_category, dim_date, fact_video_metrics
.\run_dbt.ps1 test        # -> PASS=15 WARN=0 ERROR=0 SKIP=0 TOTAL=15

# SCHRITT 7: Serverless Google Cloud Workflow in Frankfurt (europe-west3) ausfuehren
.\run_workflow.ps1        # -> Status: SUCCEEDED (0 orphan records in fact_video_metrics)

# SCHRITT 8: Grafana Cloud verbinden (Google Cloud Monitoring -> gcp_service_account.json)
#            & Dashboard importieren: dashboards/youtube_dataops_monitoring.json

# SCHRITT 9: Aufraeumen & 0-EUR-Kostengarantie nach Testende
.\stop_kafka.ps1          # Stoppt lokale Kafka-Container
.\run_infra.ps1 destroy   # Loescht alle 5 GCP-Ressourcen rueckstandslos (0,00 EUR Kosten)
```

13.3 Vollständige Referenz aller PowerShell-Runner (`.ps1`)

Runner-Skript	Parameter	Release-Status & Funktion
`run_infra.ps1`	`init`, `plan`, `apply`, `destroy`	**[v2.1 Active]** Steuert Terraform 1.17 in `infra/` (5 GCP-Ressourcen).
`start_kafka.ps1`	Keine	**[v2.0 Active]** Startet Kafka Broker (KRaft) & Kafka-UI via Docker Compose.
`stop_kafka.ps1`	Keine	**[v2.0 Active]** Stoppt Kafka-Container und gibt Ports `9092` & `8080` frei.
`run_producer.ps1`	Keine	**[v2.0 Active]** Startet `src/producer.py` (295 Clean, 31 DLQ, SCD-2 Mutationen).
`run_bronze.ps1`	Keine	**[v2.0 Active]** Konsumiert `topic_ingest` -> `data/bronze/*.parquet`.
`run_dlq.ps1`	Keine	**[v2.0 Active]** Konsumiert `topic_err` -> `data/quarantine/*.parquet`.
`run_silver.ps1`	Keine	**[v2.0 Active]** Startet Docker PySpark (`transform_silver.py`) & committet Iceberg V2.
`load_to_bigquery.ps1`	Keine	**[v2.0 Active]** Lädt Silver-Parquet idempotent nach BigQuery (`silver_videos`).
`run_dbt.ps1`	`debug`, `snapshot`, `run`, `test`	**[v2.0 Active]** Steuert dbt Core CLI (SCD-2 Snapshots, Star-Schema, 15 Tests).
`run_workflow.ps1`	Keine	**[v2.1 Active]** Triggert den Serverless Cloud Workflow via REST und pollt bis `SUCCEEDED`.
`run_gold.ps1`	Keine	*[v1.0 Legacy]* Altes Spark-Gold-Skript (in v2.0/v2.1 durch `run_dbt.ps1` abgelöst).
`start_monitoring.ps1` / `stop_monitoring.ps1`	Keine	*[v1.0 Legacy]* Lokaler Prometheus/Grafana Stack (in v2.1 durch Grafana Cloud abgelöst).
`start_airflow.ps1` / `stop_airflow.ps1`	Keine	*[v1.0 Legacy]* Lokaler Airflow 2.9 Stack (in v2.1 durch Cloud Workflows abgelöst).

13.4 Produktions-Troubleshooting Playbooks (Aus echten Fehlern gelernt)

Playbook 1: GCP IAM `Error 403: Permission denied` beim Terraform Workflow-Deployment

• *Symptom:* `terraform apply` schlägt beim Erstellen von `google_workflows_workflow` mit `403 Permission 'workflows.workflows.create' or 'iam.serviceAccounts.actAs' denied` fehl.
• *Ursache:* Zum Binden eines Service Accounts an einen Cloud Workflow benötigt der ausführende Account neben `roles/workflows.admin` zwingend auch `roles/iam.serviceAccountUser`. Versucht Terraform zudem eigene Service Accounts oder APIs zu aktivieren, scheitert dies ohne `iam.serviceAccountAdmin`.
• *Lösung:* Den existierenden Service Account (`var.service_account_email`) in `infra/main.tf` referenzieren und ihm in der GCP Console `roles/workflows.admin` sowie `roles/iam.serviceAccountUser` zuweisen.

Playbook 2: Cloud Workflows Parser-Fehler (`400 characters limit` & `multiple step types`)

• *Symptom 1:* `maximum length for an expression is 400 characters, the provided expression has a length of 743`.
• *Symptom 2:* `step 'evaluate_quality_gate': step matches multiple types: 'switch', 'assign'`.
• *Lösung:* In `infra/workflows/pipeline.yaml` lange SQL-Abfragen in mehrere Variablen (`q_target`, `q_select`, `q_from`, `q_join`) aufteilen und `assign` (`get_quality_result`) strikt vom nachfolgenden `switch`-Schritt (`evaluate_quality_gate`) trennen.

Playbook 3: Grafana Cloud Import meldet `No data sources of type cloudmonitoring found`

• *Symptom:* Beim Import des Dashboard-JSONs lässt sich die konfigurierte Google Cloud Monitoring Datenquelle nicht auswählen.
• *Ursache:* Grafana verwendet intern die Plugin-ID `stackdriver` (nicht `cloudmonitoring`).
• *Lösung:* In `dashboards/youtube_dataops_monitoring.json` ist `"pluginId": "stackdriver"` und `"type": "stackdriver"` gesetzt. Zudem benötigt der Service Account die Rolle `roles/monitoring.viewer`.

14. Data Engineering Interview Masterclass & Technical Defense (15 Q&As)

Dieses Kapitel dient als Vorbereitungsleitfaden für technische Architektur-Reviews und Vorstellungsgespräche auf Senior- und Lead-Data-Engineer-Niveau. Es liefert präzise Antworten auf 15 tiefgehende Fachfragen.

F1: Warum haben Sie sich für Apache Iceberg V2 anstelle von reinem Parquet entschieden?

Parquet ist lediglich ein Dateiformat, keine Tabellen-Abstraktion. Lose Parquet-Dateien bieten keine ACID-Transaktionen (Gefahr von Dirty Reads bei Jobabbrüchen), kein Time-Travel und erfordern teure Directory-Listings. Apache Iceberg V2 verwaltet Metadaten hierarchisch (`v1.metadata.json` -> `snap-*.avro` -> `*-m0.avro`), garantiert atomare Commits per Compare-and-Swap und ermöglicht Data Skipping über Spaltenstatistiken.

F2: Warum haben Sie Google Cloud Workflows statt Apache Airflow (Cloud Composer) gewählt?

Eine bewusste FinOps- und Architekturentscheidung: Cloud Composer benötigt einen dauerhaft laufenden GKE-Cluster (~300 €/Monat), selbst wenn die Pipeline nur wenige Minuten am Tag läuft. Da unsere Transformationslogik vollständig in BigQuery liegt (ELT), benötigen wir lediglich eine leichtgewichtige State-Machine zur Orchestrierung. Cloud Workflows startet in Millisekunden, ist 100% serverlos und kostet im Always Free Tier exakt 0,00 €.

F3: Wie handhaben Sie Slowly Changing Dimensions (SCD-2) und warum reicht SCD-1 nicht?

Bei SCD-1 überschreibt eine Titeländerung (wie bei Video `2kyS6SvSYSE`) den alten Wert — historische Analysen über den Effekt von Titel-Mutationen auf Klickraten sind damit unmöglich. Unser dbt Snapshot (`dim_video_snapshot`) implementiert SCD-2 (`strategy='check'`), schließt alte Versionen mit `valid_to` ab und legt eine neue Zeile mit `is_current = true` an. In `fact_video_metrics` ordnet ein Point-in-Time Join jedes Event exakt der damals gültigen Version zu.

F4: Was ist der Unterschied zwischen den 15 dbt-Tests und dem In-Workflow Quality Gate?

Die 15 dbt-Tests (`schema.yml`) prüfen beim Build-Prozess alle 4 Core-Marts auf `unique`, `not_null` und `relationships`. Das In-Workflow Quality Gate (`pipeline.yaml`) ist dagegen ein Laufzeit-Wächter direkt in der Cloud-Orchestrierung: Nach jedem Refresh von `fact_video_metrics` zählt ein `LEFT JOIN`-Check verwaiste Fremdschlüssel und wirft bei `invalid_count > 0` sofort eine Exception (`raise`).

F5: Wie verhindern Sie, dass fehlerhafte Streaming-Nachrichten die Pipeline zum Absturz bringen?

Durch das Dead-Letter-Queue-Muster (DLQ): `src/producer.py` injiziert 10% korrupte Nachrichten (`views = 'CORRUPT_INVALID_NUMBER'`), leitet diese aber sofort in das Quarantäne-Topic `topic_err` um, während die 295 validen Events über `topic_ingest` ungestört verarbeitet werden. `src/consumer_dlq.py` persistiert die 31 Fehler-Events samt `_error_reason` unter `data/quarantine/`.

F6: Warum wird das Zielvideo `2kyS6SvSYSE` im Producer von der 10% Fault-Injection ausgenommen?

Um 100% deterministische, reproduzierbare End-to-End-Tests für SCD-2 zu garantieren (`if vid != SCD2_TARGET_VIDEO_ID and random.random() < ERROR_RATE`). Würde eine der drei Titel-Mutationen zufällig in der DLQ landen, wäre die SCD-2-Versionskette nicht deterministisch testbar.

F7: Warum verbinden sich die Kafka-Skripte mit `127.0.0.1:9092` statt `localhost:9092`?

Unter Windows 10/11 löst `localhost` bevorzugt auf IPv6 (`::1`) auf. Wenn der Docker-Port-Forwarder auf IPv4 bindet, verursacht jeder Verbindungsaufbau einen 2-sekündigen TCP-SYN-Timeout vor dem Fallback auf IPv4. `127.0.0.1:9092` erzwingt sofortiges IPv4 ohne Disconnect-Lag.

F8: Warum führt `src/transform_silver.py` einen Dual-Write (Iceberg V2 + Parquet) durch?

Die Iceberg-Tabelle `local.silver.videos` liefert das volle Open-Table-Format mit Snapshot-Historie und ACID-Garantien im Lakehouse. Der zusätzliche Parquet-Export nach `/app/data/silver` dient als schlanker Transfer-Staging-Puffer für den lokalen Upload-Client `src/cloud_loader_bigquery.py` nach BigQuery.

F9: Warum ist der Broadcast-Join in PySpark so entscheidend für die Performance?

Ein Standard-SortMergeJoin müsste sowohl die großen Video-Events als auch die Kategorien über das Netzwerk nach `category_id` shuffeln. Da `US_category_id.json` winzig ist (< 50 KB), sendet `broadcast(df_categories)` die Lookup-Tabelle einmalig in den RAM jedes Executors. Der Join erfolgt rein lokal im Speicher (Zero Shuffle).

F10: Wie garantieren Sie Idempotenz über alle Pipeline-Stufen hinweg?

1. Silver Iceberg: `.writeTo('local.silver.videos').createOrReplace()` ersetzt den Snapshot atomar. 2. BigQuery Loader: `WriteDisposition.WRITE_TRUNCATE` überschreibt `silver_videos` ohne Duplikate. 3. dbt Core: Tabellen werden als `CREATE OR REPLACE TABLE` mit deterministischen MD5-Schlüsseln (`metric_key`) gebaut. 4. Cloud Workflow: Aktualisiert `fact_video_metrics` atomar per `CREATE OR REPLACE TABLE`.

F11: Warum haben Sie in `pipeline.yaml` das SQL-Statement in vier Variablen aufgeteilt?

Google Cloud Workflows besitzt ein hartes Parser-Limit von 400 Zeichen pro `${...}`-Ausdruck. Ein einzelner Ausdruck mit 743 Zeichen wird beim Deployment abgelehnt. Durch die Aufteilung in `q_target`, `q_select`, `q_from` und `q_join` bleibt jeder Teilstring unter 400 Zeichen und wird erst in `${q_target + q_select + q_from + q_join}` zusammengefügt.

F12: Wie optimieren Sie BigQuery-Kosten bei wachsendem Datenvolumen (Terabyte-Scale)?

Im On-Demand-Tarif berechnet BigQuery nach gescannten Bytes. Bei großen Datenmengen partitionieren wir `fact_video_metrics` nach `trending_date` (`PARTITION BY trending_date`) und clustern nach `category_key, video_key` (`CLUSTER BY`). Zusätzlich überwacht unser Grafana-Dashboard live die Metrik `bigquery.googleapis.com/query/scanned_bytes_billed`.

F13: Warum verwendet man Surrogate Keys (`video_key`) statt Natural Keys (`video_id`) im Star-Schema?

Weil `video_id` in einer SCD-2-Dimension (`dim_video`) nicht mehr eindeutig ist: Dasselbe Video hat bei 3 Titelversionen 3 Zeilen mit derselben `video_id`. Erst der Surrogate Key (`video_key = dbt_scd_id`) identifiziert exakt die historische Version, auf die die Faktentabelle verweist.

F14: Wie funktioniert CI/CD in einer echten Enterprise-Produktionsumgebung (GitOps)?

Kein Entwickler führt in Produktion manuell `dbt run` oder `terraform apply` aus. Änderungen laufen über Pull Requests: GitHub Actions authentifiziert sich keyless per OIDC (Workload Identity) gegen GCP, führt `terraform plan` und einen dbt Slim-CI-Lauf (`state:modified+`) in einem isolierten CI-Dataset aus. Erst nach Code-Review und Merge auf `main` rollt die CD-Pipeline die Änderungen automatisch nach `prod` aus.

F15: Welche Komponenten aus Release v1.0.0 sind in Release v2.1.0 obsolet und warum?

1. Der lokale Airflow-Stack (`docker-compose-airflow.yml`, `dags/`) wurde zu 100% durch serverlose Google Cloud Workflows ersetzt. 2. Der lokale Prometheus/Grafana-Stack (`docker-compose-monitoring.yml`) wurde durch Grafana Cloud ('Weg 1' direkt auf GCP Monitoring) ersetzt. 3. Das alte Spark-Gold-Skript (`src/transform_gold.py`) wurde durch das dbt Kimball Star-Schema in BigQuery ersetzt.

15. FinOps, Kosten-Engineering & Always-Free-Tier Kapazitätsplanung

Kostenoptimierung (**FinOps**) ist eine Kernverantwortung moderner Data Architects. Die gesamte YouTube DataOps Plattform (`v2.1.0`) ist so konzipiert, dass sie dauerhaft innerhalb des **Google Cloud Always Free Tiers** und **Grafana Cloud Free Tiers** betrieben werden kann.

15.1 Nachweis der 0,00 € Betriebskosten (Free Tier Kontingente vs. Verbrauch)

Cloud-Dienst	Monatliches Free-Tier-Kontingent	Realer Projekt-Verbrauch	Effektive Kosten
Google BigQuery	10 GB aktiver Speicher + 1 TB Abfragevolumen pro Monat frei.	< 5 MB Speicher (295 Zeilen) / < 100 MB Queries pro Monat.	**0,00 € / Monat**
Cloud Storage (GCS)	5 GB Standard-Storage frei; minimale Gebühren in EU bei Kleinstmengen.	Bucket `lakehouse-youtube-...` (< 10 MB; via `destroy` jederzeit löschbar).	**0,00 € / Monat**
Cloud Pub/Sub	10 GB Nachrichtenvolumen pro Monat kostenlos.	< 5 MB für `topic-youtube-ingest` & `topic-youtube-dlq`.	**0,00 € / Monat**
Cloud Workflows	5.000 interne Workflow-Schritte pro Monat kostenlos.	8 Schritte pro Workflow-Lauf (~240 Schritte bei täglichem Lauf).	**0,00 € / Monat**
Cloud Monitoring	Alle Standard-GCP-Dienstmetriken (`workflows`, `bigquery`, `pubsub`) kostenlos.	Ausschließliche Nutzung kostenloser GCP-Systemmetriken.	**0,00 € / Monat**
Grafana Cloud	Free Forever Plan: 3 User, 10k Metriken, 50 GB Logs inklusive.	1 User, 1 Dashboard (`youtube_dataops_monitoring.json`).	**0,00 € / Monat**

15.2 Implementierte Cost Guardrails (Schutz vor Kostenfallen)

- **Terraform Teardown-Garantie (`run_infra.ps1 destroy`):** Durch `force_destroy = true` am GCS-Bucket und `delete_contents_on_destroy = true` am BigQuery-Dataset lassen sich sämtliche Cloud-Ressourcen jederzeit mit einem einzigen Befehl rückstandslos entfernen.
- **Grafana FinOps-Schwellenwerte:** Panel 4 (`BigQuery Bytes Billed`) schlägt bei über 100 MB auf Orange und bei 1 GB auf Rot um, sodass teure Abfragen sofort sichtbar werden.
- **Verzicht auf 24/7-Cluster:** Weder für Compute (ephemerer Docker-PySpark) noch für Orchestrierung (Serverless Cloud Workflows) oder Monitoring (Grafana Cloud Weg 1) laufen dauerhafte VMs oder Kubernetes-Cluster.

Schlusswort: Dieses Repertorium dokumentiert den vollständigen, verifizierten Stand der **YouTube DataOps & Analytics Platform (2026 Edition)** über alle drei Releases (`v1.0.0`, `v2.0.0`, `v2.1.0`). Alle beschriebenen Skripte, Modelle, Tests, Terraform-Ressourcen, Cloud Workflows und Dashboards stimmen zu 100% mit dem produktiven Code im Repository überein.